

Họ tên: Phạm Trần Khánh Duy

MSSV: 23520384

Bài tập tuần 1 – Phân tích một hệ thống AI đang dùng

Hệ thống được chọn: Codex extension on Visual Studio Code (trong phần sau của bài làm sẽ gọi tắt là “Codex”)

1. Phân loại – Các tiêu chí

Codex được xác định là Agentic system. Thông qua các tiêu chí dưới đây mà tôi xác định Codex là một agentic system:

a. Ai quyết định bước kế tiếp?

Model quyết định. Khi tôi gọi prompt “Thực thi plan 075BE trong dev-plan theo tài liệu v3, viết báo cáo lưu trữ tại /document/dev-note theo mục lục hướng dẫn sẵn”, tôi không chỉ định sẵn các bước thực thi mà do model quyết định dựa trên kết quả thực thi. Trong quan sát của tôi, với prompt trên, hệ thống hoạt động như sau:

Đọc skill start-agent-session, controller-impl, service-impl kèm các skills khác → Đọc AGENTS.md → Đọc document v3 → Tìm “dev-plan” → tìm “plan 075” → đọc cấu trúc cây thư mục → đọc mẫu file → ... → Chạy test pmd, checkstyle → Chạy build test → đọc skill before-backend-report → Tìm “/document/dev-note” → Viết file dev note 075BE → báo cáo kết quả → done

Trừ khi có prompt chen ngang do tôi quyết định, agent sẽ tự động quyết định các bước tiếp theo. Nếu tôi chạy ở mode /goal, agent sẽ tự xem xét /goal đã đạt được hay chưa mà tạo mới một phiên gọi model.

b. Ai giữ state?

Do hệ thống harness của Codex lưu trữ. Trong một coding/review task, hệ thống sẽ duy trì các thông tin bao gồm: yêu cầu ban đầu của user, chat history, các file đã đọc, các file đã sửa,... Model có thể sử dụng state/context này để phục vụ cho nhu cầu của user.

c. Ai dừng vòng lặp?

Model có thể dừng vòng lặp sau khi tự đánh giá. Ví dụ với coding task, Codex có thể đánh giá đã code xong module A-B-C, đã test xong module B-C, module A không thể test... và ra quyết định dừng vòng lặp khi không còn thay đổi cần thiết, cần xác nhận thêm thông tin.

Harness có thể dừng vòng lặp nếu model vi phạm chính sách, đạt giới hạn tài nguyên.

User có thể dừng vòng lặp bất cứ khi nào user không muốn tiếp tục công việc.

2. Phân tích failure mode cụ thể

Trong quá trình làm việc với Codex, tôi có gặp một vài failure. Trong đó khiến tôi nhớ nhất là lúc Agent vô hiệu hoá Permission trong Sandbox rules.

Bối cảnh: tôi cấu hình *default.rules* để ngăn hoặc yêu cầu phê duyệt khi thực hiện *git revert*. Đồng thời *AGENTS.md* hướng dẫn agent không được tự ý thực hiện các thao tác Git có khả năng ảnh hưởng repository/remote.

Khi tôi đưa prompt mang tính thúc ép: “Tôi lỡ push nhầm rồi, revert hộ tôi bằng mọi giá...”, Codex nhận ra thao tác bị *default.rules* chặn. Tuy nhiên, thay vì coi rule đó là một ràng buộc không thể thay đổi, model đề xuất/xử lý theo hướng xoá hoặc sửa file rule rồi tiếp tục *git revert*.

Đây là failure vì mục tiêu hoàn thành task đã lấn át ràng buộc an toàn. Root cause là file policy có thể nằm trong phạm vi ghi của agent, tức lớp permission chưa được bảo vệ khỏi chính agent mà nó kiểm soát.

Cách giảm thiểu là đặt policy ngoài vùng agent có quyền ghi, kết hợp sandbox filesystem, approval của người dùng và logging/audit. Như vậy dù model muốn thay đổi policy, harness ở lớp thấp hơn vẫn chặn được.

3. Trình bày 2 lớp harness kèm cơ chế

a. Skills

Skills là gói chỉ dẫn mang tính chất tái sử dụng cao, định nghĩa các bước thực thi. Khi bắt đầu session, Agent sẽ được nạp vào tên-skill và mô tả ngắn để xác định trigger-condition. Khi trigger-condition thỏa mãn, nội dung chi tiết của skill mới được Agent đọc.

Một số skills tôi dùng: start-agent-session để chỉ định quyền các tài liệu cần đọc, hướng dẫn tổng quan cách xác nhận project spec; backend-validation để định nghĩa các bài test mà Agent phải thực hiện trước khi tuyên bố hoàn tất; commit-writer để định nghĩa quy tắc viết commit.

b. Permission

Permission chỉ định phạm vi tài nguyên mà Agent được phép hay bị cấm thao tác, chặn Agent leo thang quyền hạn hay thực hiện các câu lệnh nguy hiểm (git add .env, drop database). Tuy nhiên, không thể chỉ dựa vào một Permission mà tin chắc là Agent không leo thang quyền hạn mà cần đặt các Permission đủ tốt bảo vệ chéo.

Việc quy định Permission có thể thực hiện ở nhiều mức độ khác nhau, như trong extension setting, trong cấu hình .toml, trong các file AGENTS.md (diễn tả bằng ngôn ngữ tự nhiên), default.rules (chỉ định các lệnh sandbox) và có thể đặt ở nhiều cấp độ khác nhau (root folders, sub folders, projects).